

Ch. 2 : Modélisation des systèmes à événements discrets : SED

1. INTRODUCTION AUX SED :

1.1 Définition :

Les systèmes à événements discret (**SED**) sont conçus par l'homme dans le but d'accomplir des tâches de façon plus ou moins autonome. Leur fonctionnement est décrit par une suite d'évolution consécutives à des événements discrets (information donnée par un capteur ou l'appui sur un bouton de commande), et sur la base d'un programme enregistré dans sa mémoire.

La famille des **SED** englobe des systèmes très variés, allant de matériels automatiques de la vie de tous les jours (télé, calculatrice, machine à laver, etc...) aux systèmes de haute technologie (robot, drone, etc...), en passant par les chaînes de production industrielles automatisées...

Les études de cas peuvent donc être très variées, et de complexités fort différentes.

Nous nous limiterons dans ce cours à présenter les outils normalisés de communication de haut niveau (compréhensible par l'Homme) permettant la modélisation fonctionnelle des **SED**.

1.2 Notion d'entrées / sorties :

Cette notion est associée à une frontière fictive entourant la **partie commande** d'un **SED**, on distingue donc :

- **Les entrées** : Informations correspondant aux événements discrets appréhendés par le SED. Ces informations proviennent soit de **capteurs**, soit de **commandes humaines**.
- **Les sorties** : Informations renvoyées par les SED. Elles permettront le pilotage des **actionneurs** (moteur, vérin, lampe, afficheur, haut-parleur, etc...).

2. CHRONOGRAMME :

2.1 Description :

Un chronogramme est une représentation graphique de l'évolution temporelle de grandeurs. De fait, le **diagramme de séquence** proposé par la norme **SysML** (voir §4.1) est une forme de chronogramme...

Mais le plus souvent, lorsque l'on parle de chronogramme, cela correspond à la représentation de l'évolution de fonctions ou variables logiques (en ordonnée) par rapport au temps (en abscisse).

A noter que si ces **évolutions** sont **logiquement liées**, on parlera plutôt d'un **diagramme de Gantt**.

2.2 Notion de front :

Les **front montant**, noté "**↑En**" ou **front descendant**, noté "**↓En**" sont apparentés à une variable logique décrivant l'évolution d'une variable ou d'une fonction logique. Ainsi, ces « front » seront vrais durant un temps Δt considéré comme infiniment court à partir du changement d'état.

2.3 Illustration :

Observons un cycle d'ouverture d'un ouvre portail automatique domestique. Ce dernier est piloté par une télécommande avec deux boutons : un interrupteur de sécurité nommé enclenchement permettant d'activer ou désactiver le portail à distance et un bouton poussoir réalisant la commande à proprement parlé. Ce système est équipé par ailleurs d'un capteur de sécurité infrarouge détectant la présence éventuelle d'un individu dans l'espace de fermeture. Le fonctionnement se résume à :

- Le fonctionnement global du portail est conditionné par une position de l'interrupteur de sécurité sur « Enclenchement » ;
- Un front montant de « commande » provoque l'ouverture ou la fermeture du portail en fonction de sa position ;
- Lors de l'ouverture, la présence d'une personne devant les cellules de détection interrompt l'ouverture, puis au bout d'un temps Δt (réglable) celle-ci reprend.
- Lors de la fermeture, la présence d'une personne devant les cellules de détection interrompt la fermeture, puis au bout d'un temps Δt (réglable) le portail se ré-ouvre.

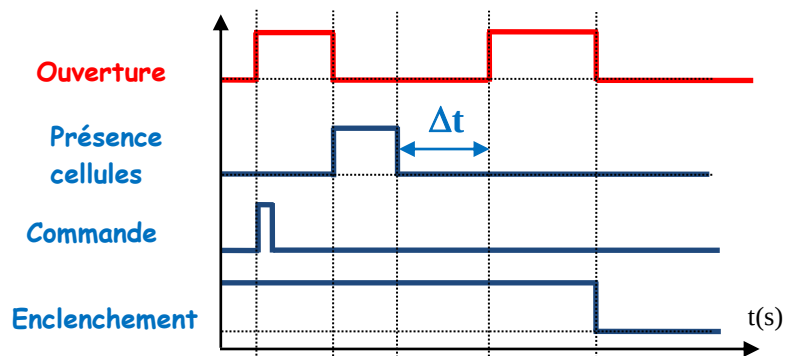


Étudions le cycle d'ouverture du portail avec un incident de présence page suivante...

On représente ci-contre le diagramme de Gant de ce cycle :

Liste des entrées et sorties du système :

Entrées	Sortie
Présence cellules	Ouverture
Démarrage	
Enclenchement	

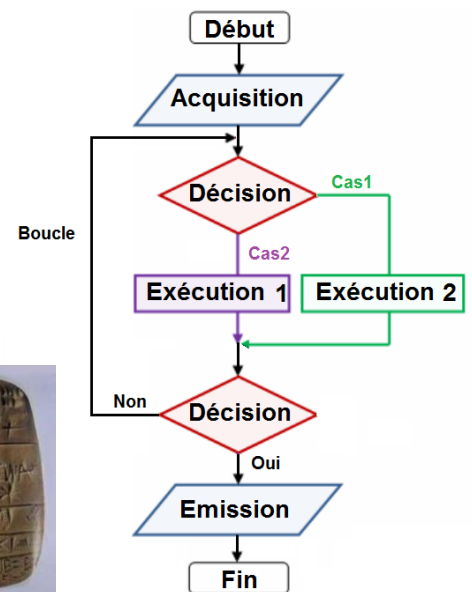


3. STRUCTURE ALGORITHMIQUE :

Un **algorithme** est une structure définissant un enchaînement logique d'activité d'un système. Il se compose d'un **début**, d'une **fin** et d'opérations élémentaires telles que l'**acquisition** de valeurs, l'**exécution** d'opérations, l'**émission** de résultats, de **décisions** et de **boucles**.

Toutes ces notions, abordées en cours d'informatique peuvent être représentées graphiquement et indépendamment de l'implémentation à l'aide de symboles illustrés ci-contre.

A noter que la norme **SysML** (voir §4.3) reprend globalement ce formalisme dans son diagramme d'activité.



Les Mésopotamiens (Irak actuelle) entre -3000 et -539 av J-C (chute de Babylone) utilisaient des algorithmes de calcul gravés sur des tablettes, tel l'exemplaire ci-contre qu'on peut observer au musée du Louvre à Paris...



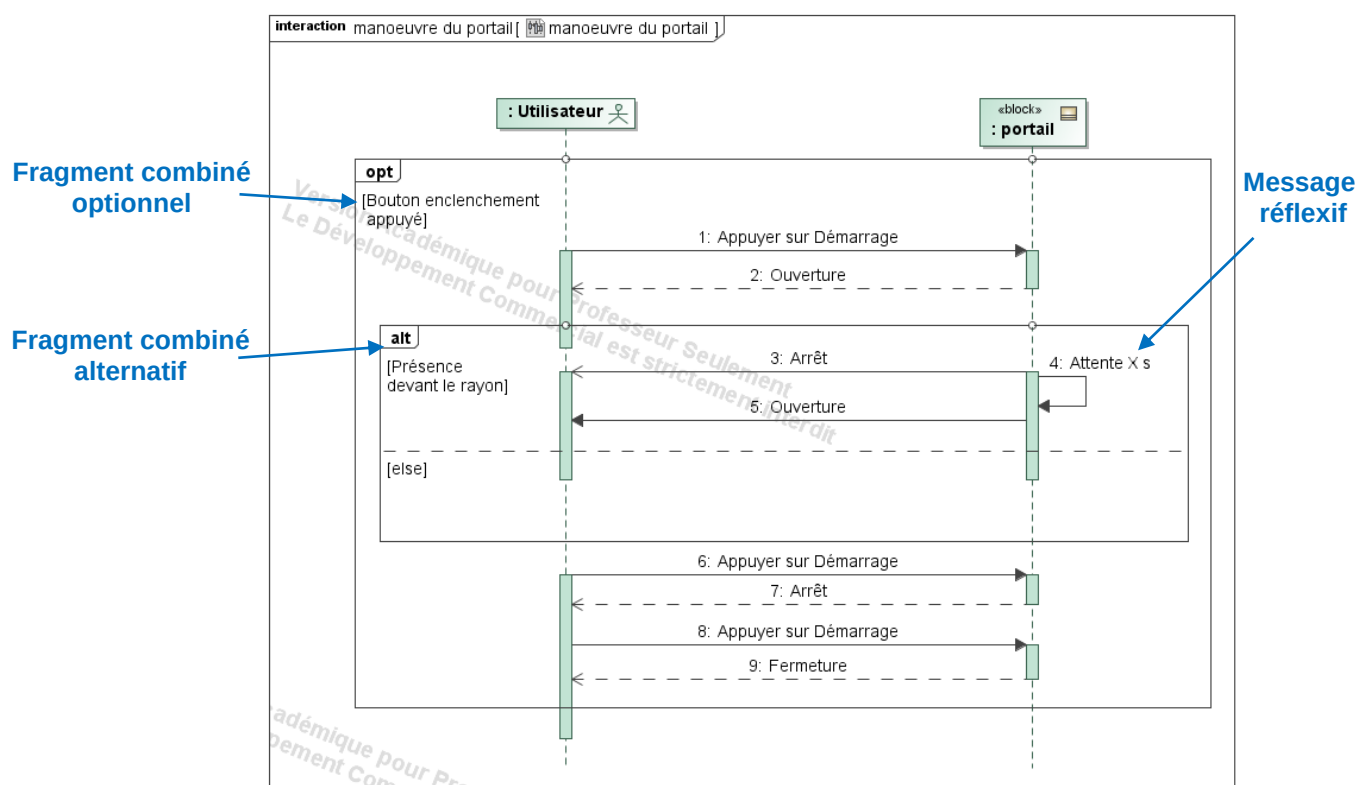
Algorithme Babylonien
Entre 1600 et 1800 av J-C

4. MODELE SYSML POUR LES SED :

4.1 Diagramme de séquence « dss » :

Ce diagramme (largement présenté en TP) définit les messages passés entre des éléments du système.

Représentons un **scénario** de fonctionnement du portail évoqué précédemment, mais cette fois-ci avec une ouverture avec un incident de présence puis la fermeture avant l'ouverture totale (sans autre incident).



Il ne s'agit là que d'un **scénario**... Tous les cas de figure ne sont pas représentés !

Le diagramme de séquence permet donc de décrire un scénario donné, ces éléments de base sont :

La ligne de vie : Représentée par une ligne verticale en pointillé, elle est associée à chaque acteur.

Message : lien unidirectionnel horizontal entre deux lignes de vie. Ils peuvent être de différents types :

- **Message synchrone** (trait plein, flèche pleine) : L'émetteur est bloqué en attente de réponse.
- **Message retour** (trait en pointillés) : Consécutif à la réception d'un autre message.
- **Message asynchrone** (trait plein, flèche évidée) : L'émetteur n'attend pas de réponse.
- **Message réflexif** (flèche bouclée) : Représente un comportement interne.

Activité : Représentée par un rectangle allongé à la verticale sur une ligne de vie, elle représente une période d'activation. Il n'est pas obligatoire de les représenter.

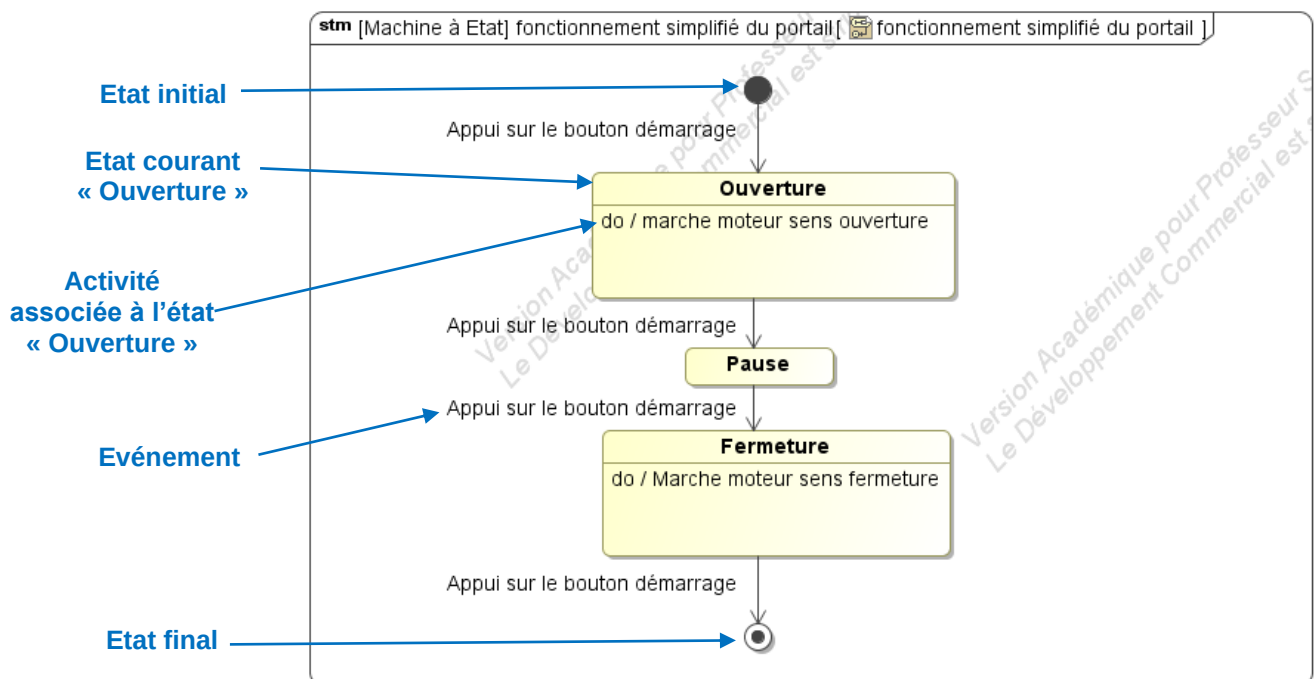
Fragment combiné : Représenté par un rectangle englobant une partie de la séquence. Chaque fragment possède un opérateur, les trois principaux sont :

- **loop** : le fragment peut s'exécuter plusieurs fois, et la condition de garde explicite l'itération.
- **opt** : optionnel, le fragment ne s'exécute que si la condition est remplie.
- **alt** : alternatif, seul le fragment possédant la condition vraie s'exécutera.

4.2 Diagramme d'état « stm » :

Typiquement, un système à événements discrets se caractérise par les différents états qu'il prend au cours du temps et des conditions d'évolution entre ces états. A ce titre, le diagramme d'état proposé par la norme **SysML** est tout indiqué pour représenter le comportement d'un **SED**.

Afin d'appréhender progressivement ce diagramme, on se propose de représenter dans un 1^{er} temps, le fonctionnement simplifié du portail (pas d'incident de présence) et sans tenir compte de l'enclenchement :



Le diagramme d'état se résume donc en une succession d'états-transitions. Ses éléments de base sont :

Etat : Il représente une période de vie de l'élément, on distingue quatre types d'états :

- **Etat initial** : pseudo-état représenté par un cercle plein, il définit le point d'entrée dans ce diagramme.
- **Etat final** : pseudo état représenté par un cercle avec un point en son centre, il est optionnel ; s'il existe, il indique la désactivation de l'état précédent. Un diagramme d'état peut contenir plusieurs états finaux.
- **Etat courant** : Représenté par un rectangle aux coins arrondis, il correspond à une activité ou à l'attente d'un événement. Plusieurs états courants peuvent être actifs en même temps.
- **Etat composite** (voir diagramme suivant) : Aussi appelé super état, un état composite englobe plusieurs sous-états. Il est donc représenté par un rectangle aux coins arrondis englobant d'autres états.

Transition : Représentée par une flèche reliant deux états, elle indique les possibilités d'évolution. En général, une transition possède un événement déclencheur (indiqué à côté de la flèche), auquel peut être associé une condition de garde et/ou un effet. Une transition n'est évaluée que si l'état précédent est actif, elle se déclenche alors, que si l'événement intervient et que la condition de garde est vraie. Le déclenchement lance l'effet associé.

Événement : Indiqué à côté d'une transition, il se produit à une date précise. Il en existe plusieurs sortes :

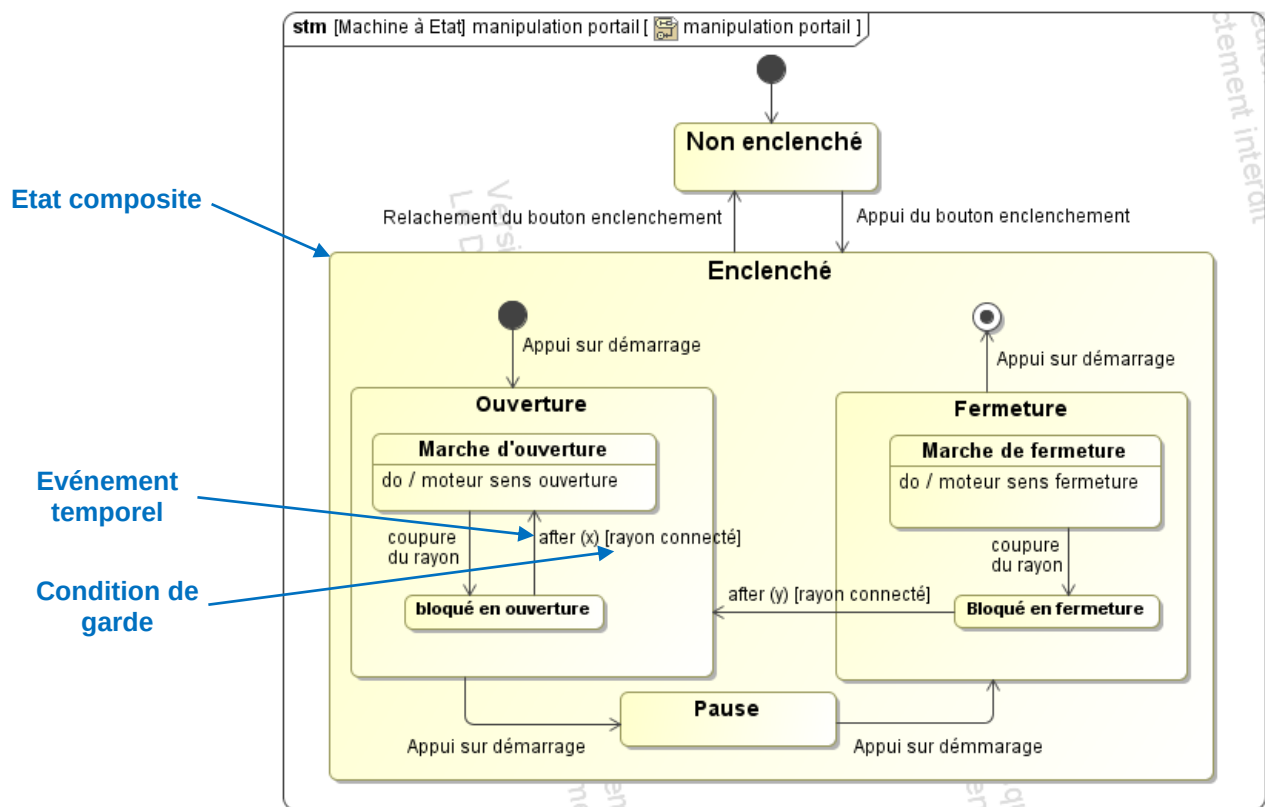
- **Événement signal** : Un signal est émis (ex : appui sur un bouton, signal capteur).
- **Événement temporel** (voir diagramme suivant) : Lié au déroulement du temps il se modélise en utilisant les mots clés « after » ou « before », suivi d'une durée.
- **Événement interne** : Associé à un état, il se modélise en utilisant le mot clé « when », suivi d'une expression booléenne.

Action : Inscrite à l'intérieur de l'état, elle s'exécute à l'activation de ce dernier. Elles peuvent être de deux types :

- **Continue** : Elle ne cesse que lorsqu'un événement provoque la désactivation de l'état.
- **Finie** : Elle cesse naturellement au bout d'un certain temps ; mais elle cesse de toute façon à la désactivation de l'état.

Les éléments de base étant définis, complétons notre diagramme en rajoutant la condition du bouton enclenchement et la possibilité d'incident de présence à l'ouverture et à la fermeture.

Notons que l'incident de présence n'aura pas le même effet en ouverture qu'en fermeture...



On précise dans ce diagramme que la coupure du rayon entraîne le blocage du système, et qu'après un temps (x ou y réglable), et à condition que le rayon soit connecté, l'état marche peut reprendre.

On voit également apparaître de nouveaux éléments : Etat composite et événement temporel, définis précédemment, mais aussi la notion de condition de garde :

Condition de garde : Indiquée entre crochets à la suite d'un événement, elle est définie par une expression logique. Elle est évaluée lorsque l'événement associé est actif. Elle conditionne l'évolution de la transition. Un même événement peut provoquer différentes évolutions suivant les conditions de garde. Il est alors important que ces conditions soient complémentaires. Pour éviter les ambiguïtés, on peut notamment utiliser la condition **[else]**.

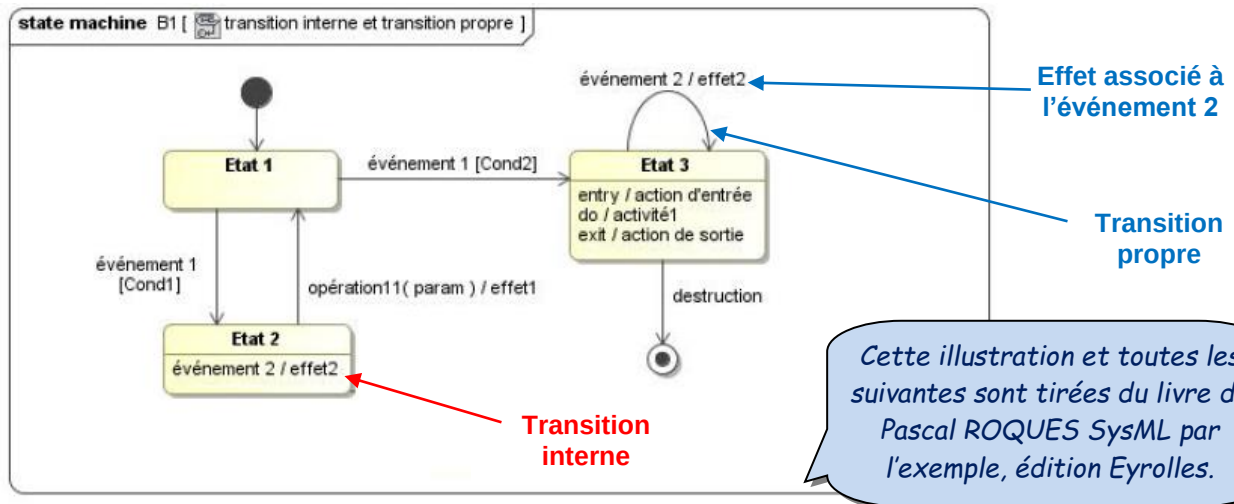
Plusieurs notions fondamentales ne sont pas abordées dans notre exemple, notons les principales (illustrées page suivante) :

Effet : Associé à une transition, de syntaxe « événement/effet », un effet est lié au déclenchement d'une transition. L'état cible est activé lorsque l'effet est terminé.

Effet d'entrée et de sortie : Associé à un état, ils correspondent à des actions qui s'exécutent respectivement à chaque activation et désactivation de l'état. Ils seront précisés à l'intérieur du symbole d'état par les syntaxes respectives : **entry/action associée** et **exit/action associée**.

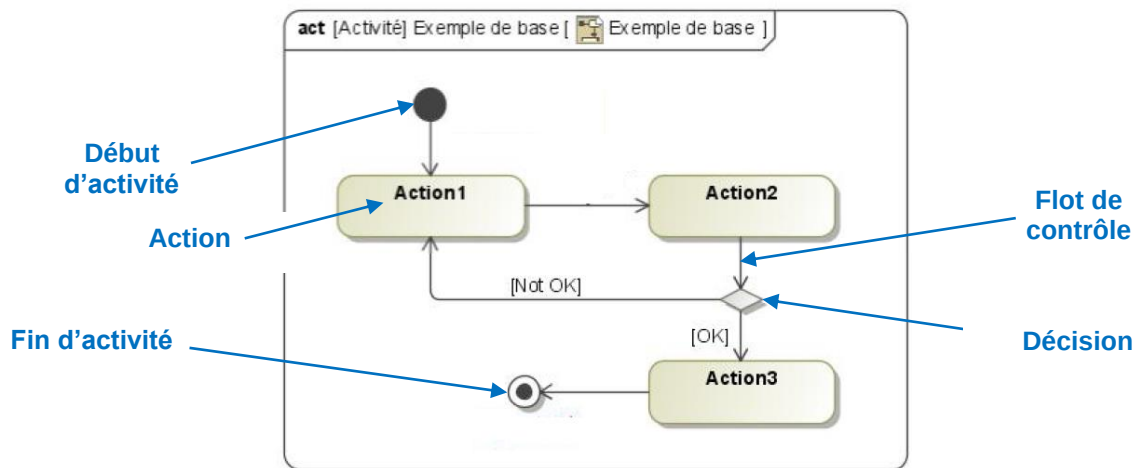
Transition propre : Elle est représentée par une flèche réflexive sur l'état. Comme une transition normale, on lui associe en générale un effet, et c'est d'ailleurs sa principale utilisation. Mais elle entraîne un effet secondaire qui est la désactivation suivie de la réactivation immédiate de l'état. Cela aura pour conséquence le relancement des activités liées à l'état bien sûr, mais surtout des activités d'entrée et de sortie, comme illustré ci-dessous.

Transition interne : Si l'on désire éviter cela, on peut aussi utiliser une transition interne, qui elle, n'aura aucun effet sur l'état en cours. Elle est notée à l'intérieur de l'état.



4.3 Diagramme d'activité (act) :

Le diagramme d'activité est un outil puissant permettant de décrire tout autant un flux de contrôle que de données au travers d'une succession d'activités. Il peut être utilisé pour décrire ce qui se passe quand le système se trouve dans un état ou pour décrire l'ensemble d'un processus. Précisons que ce diagramme est hors programme ; nous nous limiterons donc à le présenter à titre d'information. Notons toutefois que dans son expression la plus simple, il représente un algorithme graphique, lui bien au programme...



Si la syntaxe est très proche de celle utilisée dans le diagramme d'état, il ne faut pas confondre états et actions...

Action : Représentée par un rectangle aux coins arrondis, l'action est l'unité fondamentale de ce diagramme. Elle représente un traitement ou une transformation contenue dans une activité.

Flot : Représenté par une flèche il permet de faire le lien entre deux nœuds (actions, décision, etc..).

Décision : Représentée par un losange, c'est un nœud de contrôle définissant un choix dynamique entre plusieurs conditions exclusives indiquées entre crochets à côté des flots concernés.

Dans le diagramme précédent, l'activité démarre avec l'action 1. Lorsque cette dernière se termine, elle produit « un jeton » qui va alimenter l'action 2, via le flot de contrôle. A la fin de l'action 2, cette dernière produit à son tour un « jeton » qui arrive jusqu'à la décision. Suivant la condition OK ou Not OK, le « jeton » va activer l'action 3 ou l'action 1. Si c'est l'action 3 qui est activée, elle s'exécute et sa terminaison fait également terminer l'activité englobante. Si c'est l'action 1, on observe alors une boucle action 1, action 2, décision et ainsi de suite...



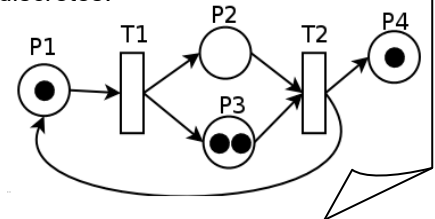
Karl Adam PETRI
1926-2010

*Mathématicien
Allemand rattaché à
l'université de
Hambourg, il
donnera son nom à la
théorie développée
pour sa thèse...*

La notion de **jeton** provient du formalisme des **réseaux de Petri** (appelé aussi Place-Transition, illustré ci-dessous), développés dans la thèse de doctorat de Karl Adam PETRI en 1962.

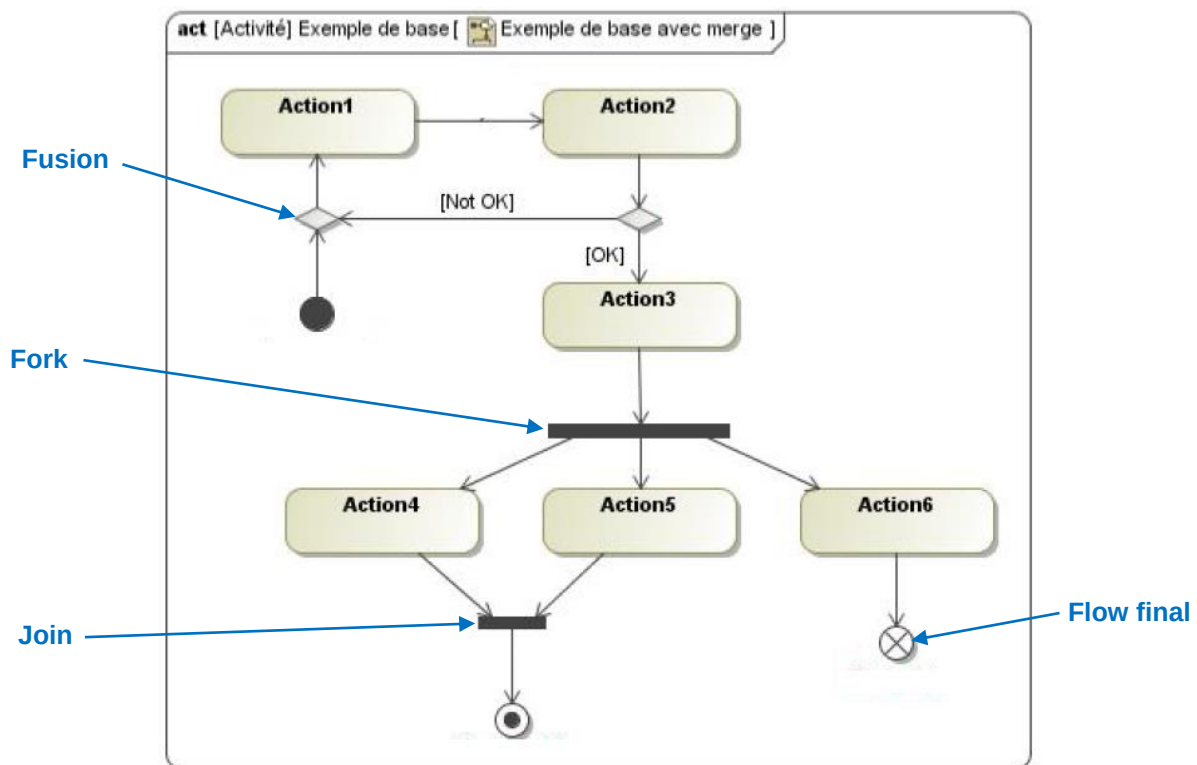
Un réseau de Petri est un modèle mathématique graphique permettant de représenter divers systèmes (industriels, informatiques, etc...) travaillant sur des variables discrètes.

Des réseaux Petri dériveront plusieurs outils de description des systèmes automatiques (comme le GRAFCET), encore utilisés dans l'industrie aujourd'hui...



Pour aller plus loin...

La décision permet donc d'envisager plusieurs cas de figure exclusifs. Si maintenant, on désire alimenter plusieurs séquences d'actions en parallèle, on aura recours au **Fork** et **Join**, décrit ci-dessous :



Fork : Représenté par une barre horizontale, il définit un nœud de débranchement en parallèle. Il possède un arc entrant et plusieurs arcs sortants.

Join : Représenté également par une barre horizontale, il définit quant à lui un nœud de synchronisation entre actions (rendez-vous).

Flow final : Contrairement à la fin d'activité qui est globale, la fin de flot est locale et n'a pas d'effet sur l'activité englobante. Elle est représentée par une croix encadrée.

Fusion : Représentée comme la décision (losange), elle est l'inverse de cette dernière. Elle possède plusieurs flots entrant pour un seul sortant et permet à une action de pouvoir être activée par différents flots.

Dans ce dernier diagramme, l'activité démarre avec l'action 1, qui peut également être activée via la fusion provenant de la boucle Action 2, décision. La terminaison de l'action 3 produit un jeton qui se divise au niveau du « Fork ». Comme le « Fork » possède trois flots de sortie, cela signifie que trois jetons indépendants existeront alors en parallèle. Ces jetons vont activer respectivement les actions 4, 5 et 6 qui vont démarrer exactement en même temps et s'exécuter indépendamment.

Toutefois, la fin d'activité n'aura lieu que lorsque le « join » aura produit son jeton. Or un « join » ne produit son jeton de sortie que lorsque tous ses flots d'entrée possèdent leur jeton. Il faut donc que les deux actions 4 et 5 soient terminées (dans un ordre quelconque) pour que l'activité englobante se termine.

Contrairement aux actions 4 et 5 qui sont synchronisées, l'action 6 est indépendante. Sa terminaison n'a aucun impact sur le reste de l'activité. Par contre, la terminaison conjointe des actions 4 et 5 provoquera la fin de l'activité et donc de l'action 6, même si celle-ci n'est pas terminée.